Experiment 5
Loops and Jump Statements

Aim / Objective:

The objective of this experiment is to understand and implement loops and jump statements in C programming. Loops such as for, while, and do-while are essential for performing repetitive tasks efficiently, reducing code redundancy, and improving program logic. Through this experiment, students will learn how to construct and apply different types of loops to execute a set of instructions multiple times based on specified conditions. Additionally, the experiment aims to introduce the use of jump statements like break, continue, and goto, which are used to alter the normal flow of control within loops or switch cases. By practicing these concepts through hands-on coding, students will enhance their ability to write efficient, readable, and logically sound programs. This experiment also helps in developing debugging skills and understanding how control statements influence the execution path of a C program…

Problem Statement:

1. Write a program to print the Fibonacci series up to N terms using a for loop.
2. Write a program to print a pattern using nested while loops.

   *

   * *

   * * *

   * * * *

   * * * * *

3. Write a program to demonstrate break in a loop when the number 13 is entered (terminate loop early).

Theory / Concept:

In C programming, **loops** and **jump statements** are fundamental control structures used to manage the flow of execution in a program.

Loops allow a set of instructions to be executed repeatedly based on a condition. There are three primary types of loops in C:

1. **for loop** – Used when the number of iterations is known beforehand. It includes an initialization, a condition check, and an update expression.
   ```
   for (int i = 0; i < 10; i++) {
      // statements}
   ```

2. **while loop –** Executes a block of code as long as the given condition remains true. It is typically used when the number of iterations is not known in advance.

```
while (condition) {
   // statements
}
```

3. **do-while loop –** Similar to the while loop, but it guarantees at least one execution of the loop body because the condition is checked after the first iteration.

```
do {
   // statements
} while (condition);
```

Jump statements are used to alter the normal flow of a program. The main jump statements in C are:

- **break –** Terminates the loop or switch statement prematurely when a specific condition is met.
- **continue –** Skips the remaining code in the current iteration and moves to the next iteration of the loop.
- **goto –** Transfers control to a labeled statement in the program. Though generally discouraged due to readability and maintainability concerns, it is still available in C.

Understanding how loops and jump statements work is crucial for writing efficient and logical programs. These structures help reduce code duplication, control program execution more precisely, and solve problems involving repetitive actions or conditional branching.

Algorithm :

Write a program to print the Fibonacci series up to N terms using a for loop.

1. Start
2. Input the number of terms N
3. Initialize the first two terms of the Fibonacci series: a = 0, b = 1
4. Print a and b
5. Repeat from i = 3 to N:
6. Calculate c = a + b
7. Print c
8. Update a = b, b = c
9. End

Write a program to print a pattern using nested while loops.

1. Start
2. Input the number of rows n
3. Initialize outer loop variable i = 1

4. While i <= n:
5. Initialize inner loop variable j = 1
6. While j <= i:
7. Print *
8. Increment
9. Print a newline
10. increment i
11. End

Write a program to demonstrate break in a loop when the number 13 is entered (terminate loop early).

1. Start
2. Loop to continuously read numbers from the user
3. For each input:
4. If the number is **13**, use break to exit the loop
5. Else, print the number
6. End the program after breaking from the loop.

Program code:

1. Write a program to print the Fibonacci series up to N terms using a for loop.

Code:

Output:

2. Write a program to print a pattern using nested while loops.

*

* *

* * *

* * * *

* * * * *

Code:

Output:

3. Write a program to demonstrate break in a loop when the number 13 is entered (terminate loop early).

Code:

Output:

Compilation and Execution Steps:

1. Open Dev C++.
2. Click File → New → Source File.
3. Type or paste your C code.
4. Click File → Save As and save the file with a .c extension.
5. Click Execute → Compile (or press **F9**).
6. If there are no errors, click Execute → Run (or press **Ctrl + F10**).
7. View output in the console window'

Learning Outcomes: Students will able to

LO2.1 Define loop control structures.

LO2.2 Illustrate the use of switch cases.

LO2.3 Illustrate the different types of ternary and conditional operators.

Course Outcome: After completion of course the students will be able to various Control Structures in C.

Practice Programs :

1. Print the following pattern using nested loops:

```
* * * * *
* * * *
* * *
* *
*
```

2. Create a menu-driven program to perform addition, subtraction, multiplication, and division using a while loop and switch.

3.Write a program to check whether a number is a palindrome using a while loop.

Conclusion:

Through this experiment, we successfully explored the use of loops and jump statements in C programming. We implemented programs using different types of loops, including for and while, to perform repetitive tasks such as generating the Fibonacci series and printing patterns. Additionally, we demonstrated the use of the break statement to control the flow of execution by terminating a loop based on a specific condition. This experiment helped us understand how loops reduce code redundancy and how jump statements provide more control over program logic. Overall, we gained practical knowledge on how to write efficient and structured programs using loops and conditional jumps in C.

Viva Questions:

1. What is a loop in C programming?
2. What are the types of loops available in C?
3. When would you use a for loop instead of a while loop?
4. What is the difference between while and do-while loops?
5. What is the purpose of the break statement?
6. What is the use of the continue statement?
7. What is the goto statement and why is it discouraged?
8. What will happen if the condition in a while loop is always true?
9. Can we use nested loops in C?
10. What happens if you place a break inside nested loops?
11. Can a loop be infinite? How?
12. What is the initial value and update part in a for loop used for?
13. Can we use more than one loop inside another loop?
14. What is the difference between break and continue?

For Faculty Use only

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	Total
Marks Obtained				